



Este é um conjunto de exercícios de revisão da disciplina **Programação Modular**. O objetivo destes exercícios é **revisar e fixar** o conteúdo estudado em aulas na disciplina. É uma atividade **não obrigatória e não pontuada**. No entanto, sua **realização é altamente recomendada** para prática da disciplina e para o levantamento de **dúvidas** e **pontos de atenção** que possam ser resolvidos com o professor.

1) Modele uma classe *Carro*. O carro é identificado por sua placa e controla sua velocidade, que deve ser limitada a 120km/h. Sua classe deve ter métodos para acelerar, frear e exibir a velocidade atual.

2) Use um diagrama de classes UML para modelar uma calculadora que consiga fazer as quatro operações matemáticas básicas. Implemente sua classe e um conjunto de testes unitários para ela.

3) Um *Livro* armazena seu título, autor e número de páginas. Ele pode receber avaliações entre 0 e 5 estrelas pelas pessoas que já o leram e, assim, calcular uma avaliação média. Além disso, se a pessoa informar quanto tempo gasta, em minutos, para ler uma página, o livro responde com uma estimativa do tempo total (em horas e minutos) para conseguir ser lido até o final. Crie um modelo UML para este problema e implemente a classe *Livro* seguindo o modelo.

4) Um *Aluno* é identificado por sua matrícula e possui um nome próprio. Ele pode fazer diversas atividades avaliativas ao longo do seu curso, todas elas valendo 10 pontos. Sua nota final é a média das avaliações. O aluno deve conseguir responder sua situação do aluno: aprovado (nota final maior ou igual a 6), em reavaliação (entre 4 e 6) ou reprovado (menor que 4). Se ele já fez reavaliação (atividade única, valendo 10 pontos), sua nota final é a média entre a reavaliação e a nota anterior do semestre. Modele esta classe em UML e implemente seu código, em conjunto com testes unitários para cada método de negócio.

5) Uma conta bancária é identificada pelo seu número de 5 dígitos e pelo CPF de seu proprietário. As contas bancárias permitem operações de saque e depósito. A operação de saque tem uma regra especial, permitindo que o cliente faça um saque acima do seu saldo – o que é chamado de *limite*. Por exemplo, se uma conta tem saldo de R\$200 e limite de R\$100, seu cliente pode sacar até R\$300. O saldo fica negativo e, no próximo depósito, a conta cobrará uma taxa de 3% sobre este valor negativo, descontando do próprio valor de depósito.

Para a situação apresentada, modele uma classe utilizando UML, implemente seu código em Java e crie dois testes unitários para cada método da classe (excetuando-se o construtor e *toString*).

6) Modele e implemente uma classe *Data*. Esta classe só pode armazenar datas válidas. Uma data deve ser impressa no formato DD/MM/AAAA e, ainda, fazer as operações:

- Somar dias à data, retornando uma nova data;
- Comparar uma data com outra, dizendo se a data está à frente da outra no calendário;
- Informar a diferença, em dias, entre duas datas.

Crie dois testes unitários para cada uma das operações acima, usando JUnit.